

A Report Submitted in Partial Fulfilment of the Requirements for

Indian Knowledge System Internship Program 2026

(sponsored by IKS Division, Ministry of Education, Government of India)

DEEP LEARNING BASED CLASSIFICATION OF INDIAN FOLK MUSIC USING AUDIO FEATURES

By

Jalagam Sirichandana

B.Tech, Computer Science and Engineering

Under the Guidance of

Dr. Ripon Patgiri

Principal Investigator

Dr. Anupam Biswas



Department of Computer Science and Engineering

NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR

Assam

Duration: 22/06/2026-21/07/2026

Declaration

I, **Jalagam Sirichandana**, student of B.Tech, Computer Science and Engineering, hereby declare that the project report titled “**Deep Learning Based Classification of Indian Folk Music Using Audio Features**” is a record of the work carried out by me under the guidance of

Dr. Ripon Patgiri as part of the Indian Knowledge Systems (IKS) Internship Program 2026.

The work presented in this report covers the development of an audio classification pipeline for six Indian folk-music traditions. It includes dataset organization, source-aware splitting, preprocessing, feature extraction, deep neural network design, class-weighted learning, feature-domain augmentation, model comparison, and held-out evaluation.

Place: Silchar, Assam

Date: 19/08/2026

Jalagam Sirichandana

Nit Silchar

CSE

2312136

Acknowledgement

I express my sincere gratitude to my mentor, **Dr. Ripon Patgiri**, for his continuous guidance, technical suggestions, encouragement, and support throughout this work. His guidance helped me approach the project as a systematic machine-learning study rather than as a single model-training exercise.

I am thankful to **Dr. Anupam Biswas**, Principal Investigator, for providing the opportunity to work on a computational project connected with Indian Knowledge Systems and for supporting an environment in which traditional cultural material could be studied using modern computational methods.

I acknowledge the IKS Division, Ministry of Education, Government of India, and the Department of Computer Science and Engineering, National Institute of Technology Silchar, for facilitating the internship and providing the academic context for this work.

I also acknowledge the open research and software ecosystem used during implementation, including Python, PyTorch, librosa, NumPy, pandas, scikit-learn, TensorBoard, and related documentation. These tools were important for building a reproducible workflow covering audio processing, model training, and evaluation.

Abstract

Indian folk music represents a diverse and culturally significant component of India's intangible heritage, with each regional tradition exhibiting distinctive rhythmic patterns, melodic structures, tonal characteristics, instrumentation, and performance practices. With the increasing availability of digital audio recordings, computational methods can support the documentation, organization, analysis, and preservation of these musical traditions. This project presents a deep-learning-based approach for the automatic classification of Indian folk music using audio signal processing and acoustic feature extraction techniques. The primary objective is to develop an end-to-end machine learning system capable of receiving an audio recording and predicting its corresponding folk-music category.

The experimental study considers six target folk-music categories: Bihu, Kamrupi_Lokgeet, Yakshagana, Rouf, Lavani, and Uttarakhandi. The dataset used in the study contains 1,807 audio clips representing 299 identifiable source songs. The recordings were organized into appropriate target categories for supervised learning, while the experimental design maintained source-level separation between training, validation, and testing data to reduce the possibility of information leakage between related audio segments. A 70:15:15 source-level split was employed, resulting in 1,267 training clips, 270 validation clips, and 270 test clips.

The audio preprocessing pipeline converts recordings into a consistent representation by resampling the audio to 22,050 Hz, converting it to mono where required, and creating fixed-duration five-second segments. To represent different aspects of the musical signal, multiple acoustic and spectral characteristics were extracted. These include Mel-spectrograms, Mel-Frequency Cepstral Coefficients (MFCCs), and chroma-based tonal features. The resulting feature representation has three channels with a shape of (3, 128, 216), allowing the audio information to be processed in a manner similar to a multi-channel image by convolutional neural networks. The extracted features provide complementary information about spectral energy distribution, timbral characteristics, and pitch-class content.

Three deep-learning architectures were investigated: a 2D Convolutional Neural Network (CNN), a CNN combined with a Bidirectional Long Short-Term Memory (BiLSTM) network, and a Convolutional Recurrent Neural Network (CRNN). These models were trained and evaluated using accuracy, precision, recall, and F1-score, with particular attention given to macro F1-score because of the differences in the number of samples available for the six categories. Class-weighted loss was also incorporated to reduce the influence of class imbalance during training.

The experiments demonstrated that the CRNN with SpecAugment provided the strongest overall performance among the tested configurations. The best-performing configuration achieved a test accuracy of 56.30% and a macro F1-score of 55.77%. The results indicate that combining convolutional feature learning with recurrent temporal modeling can provide useful representations for folk-music classification. SpecAugment further improved the model's ability to generalize by introducing controlled modifications to the extracted time-frequency representations during training. Comparative experiments also showed that increasing model complexity does not necessarily lead to better generalization, as some configurations exhibited substantial differences between training and validation performance.

An interactive Streamlit-based application was also developed as the practical interface for the trained models. The application supports audio-file upload and microphone recording and provides genre prediction together with confidence scores. In addition, it provides waveform visualization, Mel-spectrogram visualization, MFCC plots, chroma representations, and summaries of extracted acoustic features. This creates an accessible interface through which users can examine both the prediction and the underlying audio characteristics used by the system.

Overall, the project demonstrates the feasibility of applying audio signal processing and deep learning to the computational classification of Indian folk music. Although the achieved performance leaves considerable scope for improvement, the study establishes a complete workflow covering dataset organization, source-level splitting, preprocessing, feature extraction, model development, augmentation, evaluation, error analysis, and deployment through an interactive application. Future improvements can focus on expanding and balancing the dataset, obtaining expert-validated musical labels, using larger collections of authentic recordings, improving temporal modeling, applying more advanced augmentation and representation-learning techniques, and incorporating domain-specific musical knowledge. Such developments could contribute to more reliable computational tools for the documentation, analysis, preservation, and wider accessibility of India's diverse folk-music heritage.

Contents

Section	Page
Declaration	i
Acknowledgement	i
Abstract	ii
Contents	iii
List of Figures, Tables and Abbreviations	iv
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Proposed Solution	1
1.4 Objectives and Scope	1
2 Literature Survey	3
2.1 Related Work	3
2.2 Deep Learning for Audio	3
2.3 Research Gap	4
3 Proposed Work	5
3.1 Methodology	5
3.2 Dataset and Metadata	6
3.3 Preprocessing and Feature Extraction	7
3.4 Models Explored	7
3.5 Metrics	7
4 Experimental Results and Discussion	8
4.1 Experimental Setup	8
4.2 Model Comparison	8
4.3 Best Model Analysis	9
4.4 Error Analysis	10
4.5 Discussion	10
5 Conclusion and Future Work	12
References	13

List of Figures

No.	Item	Page
1	Figure 3.1	5
2	Figure 4.1	8
3	Figure 4.2	9

List of Tables

No.	Item	
1	Table 3.1	6
2	Table 3.2	6
3	Table 3.3	7
4	Table 3.4	7
5	Table 3.5	7
6	Table 4.1	8
7	Table 4.2	9
8	Table 4.3	10

List of Abbreviations

Abbreviation	Full Form
IKS	Indian Knowledge Systems
MIR	Music Information Retrieval
CNN	Convolutional Neural Network
CRNN	Convolutional Recurrent Neural Network
GRU	Gated Recurrent Unit
BiLSTM	Bidirectional Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
STFT	Short-Time Fourier Transform
F1	F1-score
BN	Batch Normalization
AMP	Automatic Mixed Precision

Chapter 1

Introduction

1.1 Background and Motivation

Indian folk music forms an important part of the country's cultural heritage. Regional traditions are shaped by local languages, communities, instruments, social contexts, and performance practices. A single national collection can therefore contain recordings with substantial variation in spectral content, rhythm, vocal style, and recording conditions.

Digital audio collections provide an opportunity to preserve and study this diversity using computational methods. Manual cataloguing remains essential, but automatic classification can act as a supporting tool for organizing large collections, finding similar recordings, and creating searchable archives. Machine learning is particularly useful when the input is represented in a form that exposes meaningful acoustic patterns.

Music Information Retrieval (MIR) provides the technical framework for extracting information from audio. Mel-frequency representations, MFCCs, and chroma features capture different aspects of sound. Combining them allows a model to observe both broad spectral energy and more compact tonal or cepstral characteristics.

1.2 Problem Statement

The problem addressed in this project is automatic classification of five-second Indian folk music segments into six target categories: Bihu, Kamrupi_Lokgeet, Yakshagana, Rouf, Lavani, and Uttarakhandi. The classifier must learn from a limited collection in which the number of available recordings is not uniform across categories.

A further challenge is that multiple segments may originate from the same source recording. If such segments are randomly divided across training and testing, the model may see nearly identical acoustic material during training and evaluation. Therefore, the project uses source-level splitting.

1.3 Proposed Solution

The proposed solution converts audio into fixed-size three-channel feature tensors and uses deep neural networks to learn discriminative patterns. A CNN provides local time-frequency feature extraction, while the CRNN additionally models temporal relationships using a bidirectional GRU. Regularization is introduced through weight decay, dropout, early stopping, class weighting, and SpecAugment.

1.4 Objectives

- Build a reproducible audio classification pipeline from metadata through held-out evaluation.
- Construct a multi-channel representation using Mel, MFCC, and chroma information.
- Compare CNN, CNN-BiLSTM, and CRNN architectures under a common experimental setup.
- Address unequal class frequencies using class-weighted cross-entropy.
- Investigate feature-domain SpecAugment and measure its effect on generalization.
- Establish a reproducible audio-classification workflow covering dataset preparation, preprocessing, feature extraction, model training, and evaluation.
- Develop and compare multiple deep-learning architectures so that the effect of convolutional and recurrent modelling can be assessed systematically.

1.5 Scope of the Work

The work focuses on supervised classification rather than music generation, transcription, or complete musicological interpretation. The model receives a fixed five-second representation and predicts one of the six target categories. This controlled setting makes it possible to compare architectures and training strategies consistently.

The project also emphasizes engineering reproducibility. Configuration is maintained in YAML, random seeding is used, feature arrays are stored explicitly, model checkpoints are saved, and training and evaluation are separated. This modular structure makes later experiments easier to reproduce and compare.

1.6 Significance for IKS

A computational classifier should not be interpreted as a replacement for cultural expertise. Instead, it can help researchers and archivists locate patterns in large collections and identify recordings for closer human analysis. In an IKS context, such tools can support documentation while keeping cultural interpretation in the hands of domain experts.

The project also demonstrates the importance of combining technical evaluation with awareness of the underlying categories. Closely related traditions can have overlapping acoustic characteristics, and performance conditions can vary considerably. Consequently, class-wise analysis is essential.

1.7 Report Organization

Chapter 2 reviews related work in machine learning for audio and music classification. Chapter 3 presents the dataset, preprocessing, feature extraction, architectures, augmentation, and metrics. Chapter 4 presents the experiments and analyses the strongest model. Chapter 5 summarizes the findings and proposes future work focused on data quality, expert validation, stronger augmentation, and improved architectures.

Chapter Summary. The chapter established the motivation and scope of the project and defined the classification problem. The next chapter reviews the computational ideas underlying the selected methods.

The remainder of this report follows this progression from background to implementation and evaluation. Chapter 4 presents the experimental results, compares the investigated configurations, analyses class-wise behaviour, and discusses the major sources of error. Chapter 5 concludes the study by summarising the main findings and identifying directions for future improvement.

The organization therefore provides a continuous link between the problem definition, the methodological choices, the experimental evidence, and the final conclusions. This structure also makes it possible to distinguish the development decisions from the quantitative results obtained on the held-out test set.

Chapter 2

Literature Survey

2.1 Related Work

Traditional music classification commonly uses engineered descriptors such as MFCCs, spectral statistics, rhythm descriptors, and tonal features. These features can be combined with classifiers such as KNN, support vector machines, and multilayer perceptrons. Such systems remain useful when the dataset is small and the feature design captures the distinctions relevant to the target categories.

Deep learning introduced the ability to learn hierarchical representations from raw or transformed audio. When audio is represented as a spectrogram, convolutional layers can process the representation similarly to an image while preserving local relationships between frequency and time. CNN-based approaches have therefore become common in audio tagging and music classification.

Recurrent architectures complement convolution by modelling sequences. In music, temporal context can be important because a short local spectral pattern may be ambiguous without information about neighbouring frames. CRNNs combine convolutional extraction with recurrent sequence modelling and are well suited to time-frequency inputs.

2.2 Indian and Regional Music Classification

Research on Assamese and Indian music has demonstrated the feasibility of machine learning for song and melody classification. Earlier work has used MFCC and other acoustic descriptors with conventional classifiers, while more recent work has explored supervised learning and neural methods. These studies show that computational approaches can capture meaningful regional patterns while also revealing the difficulty of separating musically related categories.

The present work follows this direction but focuses on a reproducible deep-learning comparison using a common feature tensor and a source-level evaluation strategy. The emphasis is on measuring the behaviour of alternative architectures and regularization settings rather than asserting universal boundaries between folk traditions.

2.3 Deep Learning for Time-Frequency Audio

A Mel spectrogram compresses frequency information onto a perceptually motivated scale. MFCCs provide a compact representation of the spectral envelope, while chroma features summarize pitch-class energy. Together, these representations provide complementary information for a neural classifier.

Time-frequency audio representations are particularly suitable for deep learning because they preserve local patterns in both frequency and time. Convolutional layers can learn filters that respond to characteristic spectral structures, while successive layers combine these local responses into higher-level representations.

For music classification, such representations can capture recurring harmonic patterns, spectral envelopes, rhythmic activity, and changes in timbre. However, the temporal dimension may contain dependencies extending beyond a single local convolutional receptive field. This motivates architectures that combine convolutional feature extraction with recurrent sequence modelling.

2.4 SpecAugment and Regularization

Small datasets present a strong overfitting risk for neural networks. A model can learn highly specific patterns from the training recordings and achieve very high training F1 while performing substantially worse on unseen sources. Regularization methods such as dropout, weight decay, early stopping, and data augmentation are therefore important.

SpecAugment is particularly convenient when features are already stored because it can mask regions of the spectral representation without returning to the raw audio. Frequency masking removes selected frequency bands, while time masking removes selected temporal frames. The model is thereby encouraged to use evidence distributed throughout the representation.

In this project, the augmentation implementation supports both audio-domain and feature-domain methods. Because the final training workflow uses pre-extracted NumPy features, SpecAugment is the active augmentation method. The feature tensor retains its original shape after masking.

2.5 Research Gap

The project addresses several practical gaps: (1) a complete and reproducible pipeline from audio features to evaluation; (2) comparison of CNN and recurrent architectures under a common representation; (3) source-level splitting to reduce leakage; (4) explicit handling of class imbalance; (5) direct evaluation of SpecAugment as a generalization strategy; and (6) detailed class-wise error analysis.

2.6 Summary

The reviewed approaches motivate the use of convolutional processing for time-frequency representations and recurrent processing for temporal context. The literature also supports the use of augmentation and regularization in small audio datasets. These principles form the basis of the methodology described next.

Overall, the literature indicates that successful music-audio classification depends on both an informative representation and an appropriate learning architecture. Time-frequency representations provide a compact way of preserving the evolving spectral structure of a recording, while convolutional networks are effective at learning local patterns within these representations.

Recurrent layers can further model temporal relationships, although greater model capacity can also increase the risk of overfitting when the available collection is limited. Data augmentation and careful train-validation-test separation are therefore important considerations. These observations motivate the experimental design adopted in the following chapter, where multiple architectures are compared under the same data and evaluation protocol.

Chapter 3

Proposed Work

3.1 Methodology

Stage	Operation	Output
1	Dataset organization	Metadata and source identifiers
2	Source-level splitting	Train / validation / test partitions
3	Audio preprocessing	Mono, 22,050 Hz, fixed 5-second segments
4	Feature extraction	Mel + MFCC + chroma
5	Feature storage	NumPy tensor (3,128,216)
6	Model training	CNN / CNN-BiLSTM / CRNN
7	Regularization	Class weights + SpecAugment
8	Evaluation	Accuracy, precision, recall, F1, confusion matrix

Figure 3.1: End-to-end classification pipeline

DATA	→	PREPROCESS	→	FEATURES	→	CRNN	→	RESULT
Audio clips	→	5 s / 22.05 kHz	→	3×128×216	→	BiGRU	→	Metrics

The implementation is modular. Configuration, feature extraction, augmentation, model construction, training, and evaluation are maintained in separate components. This separation allows the same data and evaluation protocol to be reused across different architectures.

3.2 Dataset and Metadata

The working collection contains **1,807 audio clips** across the six target traditions. Metadata records class identifiers, source identifiers, and chunk identifiers. The source identifier is important because several clips can belong to the same source recording.

The metadata records the source identifier, target class, clip identifier, and dataset split for every extracted segment. Keeping these fields explicitly associated with each feature file makes the experimental pipeline reproducible and allows predictions to be traced back to their corresponding source recording.

The working collection contains 1,807 audio clips representing 299 identifiable source songs. The six target categories are Bihu, Kamrupi_Lokgeet, Yakshagana, Rouf, Lavani, and Uttarakhandi. Because several clips can originate from the same source song, the source identifier is used when constructing the train, validation, and test partitions.

3.2.1 Class Distribution

Target Class	Clips	Source IDs
Bihu	396	66
Kamrupi_Lokgeet	402	67
Yakshagana	384	63
Rouf	241	39
Lavani	210	35
Uttarakhandi	174	29
Total	1,807	299

Table 3.1: Distribution of the working audio collection

The target categories are the six experimental classes used throughout the study. The Assamese target categories require particular care in interpretation because closely related folk traditions can share acoustic properties. The computational labels should therefore be viewed in the context of the selected recording collection and should be validated by domain experts in future work.

3.2.2 Source-Level Split

Split	Clips	Source IDs
Training	1,267	209
Validation	270	45
Test	270	45
Total	1,807	299

Table 3.2: Source-level partition used for model development

The 70/15/15 partition was performed at source level rather than at individual clip level. This is a critical methodological choice because segments from one recording can have highly similar background noise, instrumentation, loudness, and musical content.

3.3 Preprocessing and Feature Extraction

The audio pipeline uses mono audio sampled at 22,050 Hz and five-second segments. A hop duration of 2.5 seconds was used during segmentation. The fixed duration ensures that every stored feature tensor has a common temporal dimension and can be processed in batches.

The preprocessing stage standardizes the recordings before feature computation. Audio is resampled to 22,050 Hz and converted to mono where required. Recordings are divided into fixed five-second segments with a 2.5-second hop, producing a consistent temporal input for the neural networks.

Three complementary representations are then extracted from each segment. The Mel-spectrogram describes the distribution of spectral energy on a perceptually motivated frequency scale, MFCCs provide information related to the spectral envelope and timbral characteristics, and chroma features represent pitch-class content. These representations are combined into a three-channel tensor.

3.4.1 CRNN Architecture

The CRNN begins with four convolutional stages. The channel sequence is $3 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 256$. Batch normalization and ELU activations are used, with max pooling in the early stages and adaptive pooling to a fixed frequency dimension. The resulting representation is rearranged into a sequence over time.

A two-layer bidirectional GRU with hidden size 128 models the temporal sequence. The forward and backward hidden states provide context from both directions. Instead of selecting only the final recurrent step, the project uses temporal mean pooling over the recurrent outputs. A dense layer with 128 units and dropout 0.4 precedes the six-class output layer.

3.5 Training Configuration

Parameter	Value
Batch size	16
Maximum epochs	40
Learning rate	0.0003
Optimizer	AdamW
Weight decay	0.001
Scheduler	ReduceLROnPlateau
Scheduler factor	0.5
Scheduler patience	3
Early stopping patience	8
Loss	Class-weighted CrossEntropyLoss
Best augmentation	SpecAugment
Device	CPU
Random seed	42

Table 3.5: Training configuration for the optimized CRNN experiment

Class weights were calculated from the training split so that less frequent categories received higher lossweight. The resulting weights were approximately [0.765, 0.749, 0.782, 1.250, 1.408, 1.760] for the sixclasses in configuration order.

3.6 Performance Metrics

Accuracy reports the overall proportion of correct predictions. Precision measures the correctness of predictions assigned to a class, while recall measures how many examples of that class were recovered.F1-score combines precision and recall. Macro F1 averages class-wise F1 equally, making it useful whenclass sizes differ. Weighted F1 accounts for class support.

The training pipeline selects the best checkpoint using validation macro F1. After training, the best checkpoint is reloaded before the test set is evaluated. This prevents the final reported test result from depending on the last training epoch.

Chapter 4

Experimental Results and Discussion

4.1 Experimental Setup

The experiments used 1,267 training clips, 270 validation clips, and 270 held-out test clips. All models used the same feature representation and test partition. The main training configuration used batch size 16, learning rate 0.0003, weight decay 0.001, AdamW, ReduceLROnPlateau scheduling, and early stopping.

Initial implementation checks confirmed that the models accepted real feature batches of shape (16,3,128,216), generated six-class outputs, computed valid loss values, and completed backward propagation. These checks were completed before long CPU training runs.

4.2 Model Comparison

The initial CNN obtained 52.96% test accuracy and 51.99% macro F1. The CNN-BiLSTM obtained 48.89% accuracy and 50.71% macro F1, while the initial CRNN obtained 49.26% accuracy and 49.55% macro F1. These results show that adding recurrent capacity does not automatically improve generalization.

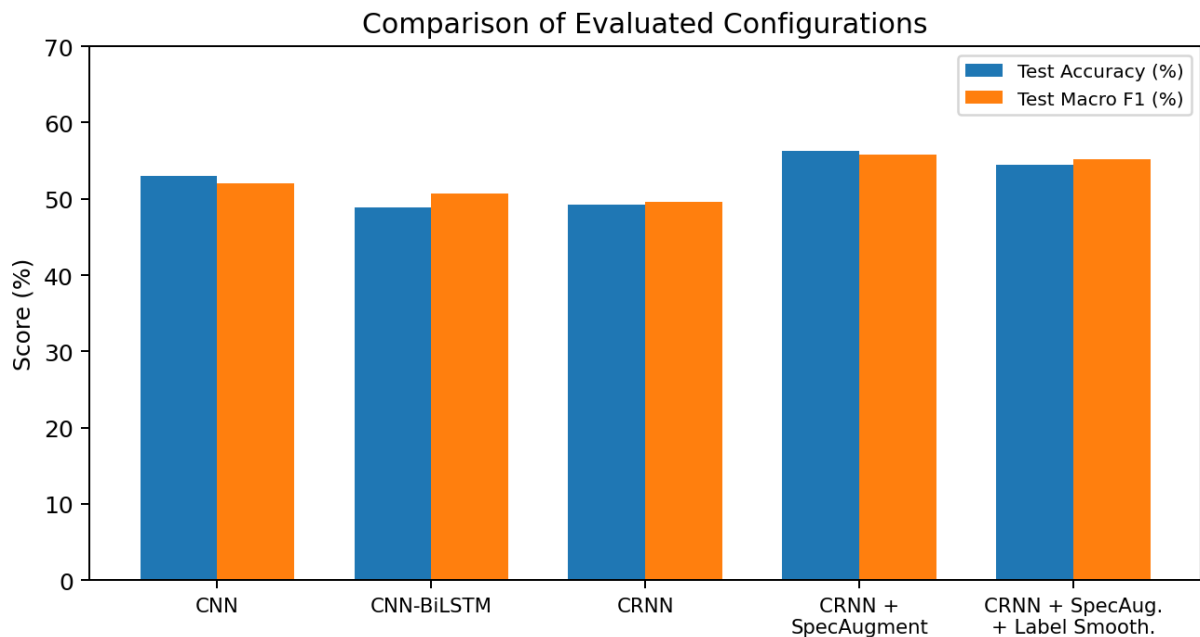


Figure 4.1: Comparison of evaluated deep learning configurations

Configuration	Test Accuracy	Test Macro F1
CNN	52.96%	51.99%
CNN-BiLSTM	48.89%	50.71%
CRNN	49.26%	49.55%
CRNN + SpecAugment	56.30%	55.77%
CRNN + SpecAugment + Label Smoothing	54.44%	55.19%

Table 4.1: Overall experimental comparison

4.3 Best Model: CRNN + SpecAugment

The strongest configuration was the CRNN with SpecAugment. Compared with the same CRNN without augmentation, test accuracy increased from 49.26% to 56.30%, an absolute improvement of 7.04 percentage points. Macro F1 increased from 49.55% to 55.77%, an improvement of 6.22 percentage points.

This experiment provides the clearest evidence from the project that regularization was beneficial. The model was trained on randomly masked feature representations while validation and test features remained unmodified.

The best validation macro F1 was 0.6145 and the best validation accuracy was 0.6222. Training still showed strong overfitting, with training F1 becoming much higher than validation F1 in later epochs. Early stopping therefore remained important.

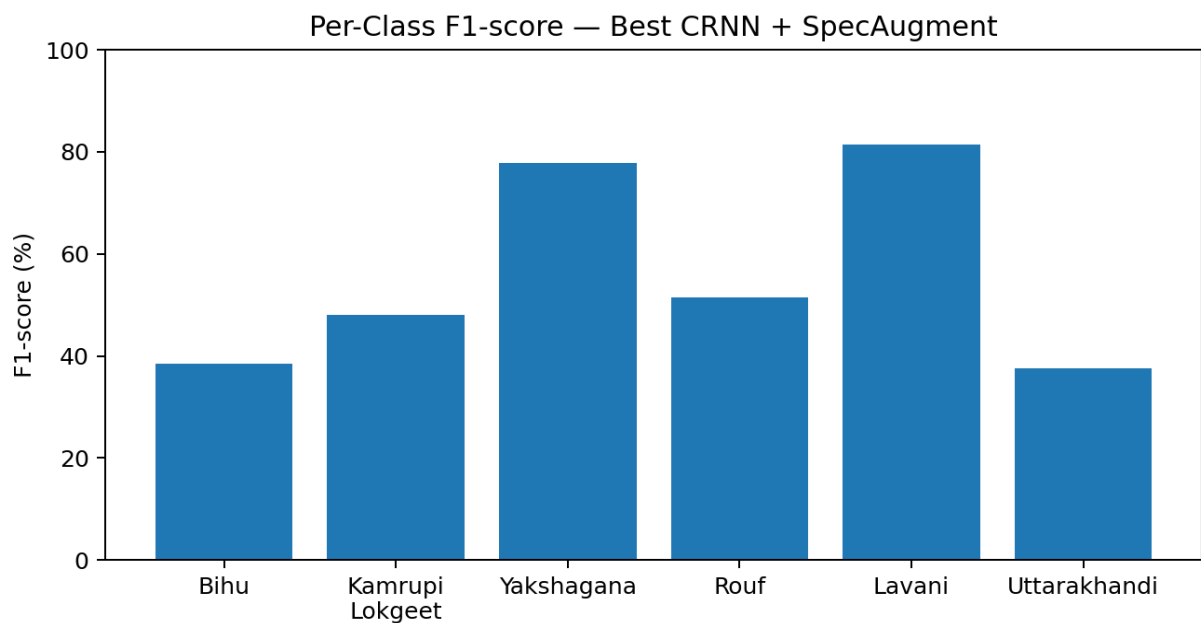


Figure 4.2: Per-class F1-score for the best CRNN + SpecAugment model

Class	Precision	Recall	F1	Support
BiHu	0.4545	0.3333	0.3846	60
Kamrupi_Lokgeet	0.4384	0.5333	0.4812	60
Yakshagana	0.7424	0.8167	0.7778	60
Rouf	0.5294	0.5000	0.5143	36
Lavani	0.8276	0.8000	0.8136	30
Uttarakhandi	0.3750	0.3750	0.3750	24

Table 4.2: Class-wise evaluation of the best configuration

4.4 Confusion Matrix and Error Analysis

The strongest class-wise performance was obtained for Lavani and Yakshagana. Lavani reached 0.8136 F1 with 80% recall, while Yakshagana reached 0.7778 F1 with 81.67% recall. These results suggest that the selected feature representation contains useful discriminative information for these categories.

Bihu and Uttarakhandi were the weakest classes, with F1-scores of 0.3846 and 0.3750. The Bihu class was particularly confused with Kamrupi_Lokgeet, indicating substantial acoustic overlap under the selected experimental labels.

Actual \ Pred.	Bihu	Kamrupi	Yakshagana	Rouf	Lavani	Uttarakhandi
Bihu	20	29	1	4	0	6
Kamrupi_Lokgeet	12	32	5	4	1	6
Yakshagana	4	2	49	1	0	3
Rouf	6	8	1	18	2	0
Lavani	0	0	1	5	24	0
Uttarakhandi	2	2	5	6	0	9

Table 4.3: Confusion matrix of the best CRNN + SpecAugment model

The confusion matrix shows that 20 of 60 Bihu examples were correctly predicted, while 29 were assigned to Kamrupi Lokgeet. For Kamrupi Lokgeet, 32 of 60 were correctly classified and 12 were predicted as Bihu. This pattern suggests that improving the representation or obtaining more reliable style-level annotation may be more valuable than simply increasing model capacity.

4.5 Label Smoothing Experiment

A label-smoothing value of 0.1 was tested with the same CRNN and SpecAugment setup. The experiment achieved 54.44% test accuracy and 55.19% macro F1, compared with 56.30% and 55.77% for SpecAugment without label smoothing. Because both test measures were lower, the label-smoothing configuration was rejected.

Label smoothing was evaluated as an additional regularization technique for the CRNN model. Instead of assigning complete probability to the target class, it distributes a small portion of the target probability among the remaining classes. This can prevent the network from becoming excessively confident in its predictions and may improve generalization on unseen audio samples.

The results suggest that the main improvement in generalization came from SpecAugment rather than label smoothing. The CRNN model was able to benefit from variations introduced in the time-frequency representation during training. Label smoothing, on the other hand, reduced the separation between the target class and competing classes. This may have affected the model’s ability to learn distinctive characteristics of closely related folk-music categories. The experiment also shows that regularization techniques need to be evaluated empirically for a specific dataset. Therefore, the final configuration retained SpecAugment while excluding label smoothing. This configuration provided the best overall test performance among the evaluated CRNN variants.

4.6 Discussion

The experiments indicate that the primary difficulty is generalization rather than optimization failure. The CRNN can fit the training set strongly, but its performance on unseen recordings is much lower. This is inconsistent with a relatively small collection of independent source recordings and with the complexity of a model containing more than two million trainable parameters.

The CNN baseline provides a useful reference because it achieved 52.96% accuracy without recurrent sequence modelling. The CRNN with SpecAugment improved upon this result, showing that temporal modelling combined with targeted regularization can be effective. The CNN-BiLSTM did not show the same advantage, suggesting that architecture complexity must be matched to data scale.

4.7 Limitations

- The number of independent source recordings is limited relative to the parameter count of the neural models.
- Class sizes are unequal, although class-weighted loss reduces the direct effect of imbalance.
- Five-second segments do not capture the full temporal structure of complete songs.
- The present feature-domain augmentation does not include the raw-audio transformations available in the augmentation module.
- Background noise and recording-quality differences can introduce features that are unrelated to the musical tradition itself.
- Closely related folk traditions can overlap acoustically, making class definitions and expert validation important.
- Experiments were run on CPU, restricting the number of hyperparameter trials that could be performed efficiently.
- The extracted MFCC and chroma representations may not capture all culturally meaningful rhythmic and melodic characteristics.

Overall finding. Within the experiments completed, the most useful intervention was SpecAugment, not additional model capacity or label smoothing. The best model is therefore retained as the CRNN + SpecAugment configuration.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project developed a reproducible deep-learning pipeline for classification of Indian folk music using audio-derived features. The pipeline includes source-aware dataset splitting, fixed-duration segments, Mel/MFCC/chroma feature construction, class-weighted loss, alternative neural architectures, SpecAugment, checkpointing, and quantitative evaluation.

The best evaluated configuration was the CRNN with SpecAugment. It achieved **56.30% test accuracy and 55.77% macro F1**. This was better than the CNN baseline at 52.96% accuracy and 51.99% macro F1, and substantially better than the CRNN without SpecAugment at 49.26% accuracy and 49.55% macro F1.

The results show that the problem remains challenging. The model performs strongly on Lavani and Yakshagana but has difficulty with Bihu and Uttarakhandi. The confusion between Bihu and Kamrupi_Lokgeet also highlights the importance of high-quality category definitions and expert validation.

The project therefore provides a useful computational baseline rather than a final automated authority on folk-music identity. Its main contribution is a working, measurable pipeline through which future datasets, features, architectures, and expert annotations can be evaluated.

5.2 Future Work

1. **Expert annotation:** Validate and refine the target labels with folk-music researchers and performers.
2. **More source recordings:** Expand the number and diversity of independent songs in each class.
3. **Raw-audio augmentation:** Carefully integrate pitch shift, time stretch, noise, and gain before feature extraction.
4. **Long-context modelling:** Aggregate predictions over multiple segments to classify complete songs.
5. **Advanced architectures:** Evaluate attention-based CRNNs, pretrained audio encoders, and lightweight audio transformers.
6. **Explainability:** Visualize important frequency-time regions and investigate why related classes are confused.
7. **Cross-dataset evaluation:** Test robustness on independent collections recorded under different conditions.
8. **Application development:** Integrate the trained model into an interactive interface for exploratory folk-music analysis.

5.3 Final Remark

Modern audio learning can support the organization and exploration of Indian folk music, but technical performance must be considered together with cultural context and annotation quality. The present work establishes a baseline and demonstrates that careful representation and regularization can materially improve generalization. Future progress should combine stronger computational models with richer datasets and domain expertise.

References

- [1] S. K. Barthakur and P. Sarma, “Automated classification of different categories of Assamese songs using KNN and MLP,” Proc. International Conference on Smart Systems and Social Management, 2025.
- [2] K. Bora, M. P. Barman, A. N. Patowary, and T. Bora, “Classification of Assamese folk songs’ melody using supervised learning techniques,” Indian Journal of Science and Technology, vol. 16, no. 37, pp. 3122–3131, 2023.
- [3] K. Choi, G. Fazekas, M. Sandler, and K. Cho, “Convolutional recurrent neural networks for music classification,” ICASSP, 2017.
- [4] K. J. Piczak, “Environmental sound classification with convolutional neural networks,” MLSP, 2015.
- [5] B. McFee et al., “librosa: Audio and music signal analysis in Python,” Proc. 14th Python in Science Conference, 2015.
- [6] Y. Wang et al., “SpecAugment: A simple data augmentation method for automatic speech recognition,” Interspeech, 2019.
- [7] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” ICLR, 2015.
- [8] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” ICLR, 2019.
- [9] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” NeurIPS, 2019.
- [10] J. Salamon and J. P. Bello, “Unsupervised feature learning for urban sound classification,” ICASSP, 2015.
- [11] S. Hershey et al., “CNN architectures for large-scale audio classification,” ICASSP, 2017.
- [12] J. F. Gemmeke et al., “Audio Set: An ontology and human-labeled dataset for audio events,” ICASSP, 2017.
- [13] C. M. Bishop, Pattern Recognition and Machine Learning. Springer, 2006.
- [14] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning. Springer, 2009.
- [15] F. Chollet, Deep Learning with Python, 2nd ed. Manning, 2021.
- [16] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” Journal of Machine Learning Research, vol. 9, pp. 2579–2605, 2008.
- [17] A. Vaswani et al., “Attention is all you need,” Advances in Neural Information Processing Systems, 2017.
- [18] scikit-learn developers, scikit-learn: Machine Learning in Python, software documentation and API reference.
- [19] NumPy developers, NumPy: The fundamental package for scientific computing with Python, software documentation and API reference.
- [20] pandas development team, pandas: Python Data Analysis Library, software documentation and API reference.

End of Report.